Cyriac Dupouey

Rapport P2I du 17/03/2026

Semaine 5 **1**
Génération d'une miniature 1
Semaine 6 **3**
Ré-adaptation de la génération du best-of 3
Complément avec les segments de scènes 4
Réorganisation chronologique des segments 5
Fusion des segments proches 5
Pour la semaine 7 **6**
Annexe **6**
Miniature 8

Semaine 5

Génération d'une miniature

Un vrai logiciel génère en général une image représentative du best-of, cela s'appelle une miniature. Pour cela, je vais prendre le moment le plus intense, extraire la frame et l'enregistrer comme thumbnail.png

1. On récupère le meilleur moment

   Etant donné que j'ai déjà trié les moments par score:

   ```
   moments = sorted(moments, key=lambda x: x[2], reverse=True)
   ```

   Le meilleur moment est : best_moment = moments[0]

2. On crée une fonction pour générer cette miniature

   ```
   def generate_thumbnail(video_path, moment):

       start_frame = moment[0]

       cap = cv2.VideoCapture(video_path)
       cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)

       ret, frame = cap.read()
   ```

```
    if ret:
        base_path = Path(__file__).resolve().parent.parent
        thumbnail_path = base_path / "outputs" / "thumbnail.png"

        cv2.imwrite(str(thumbnail_path), frame)

        print("🖼️ Thumbnail généré :", thumbnail_path)

    cap.release()
```

3. J'appelle la fonction

```
generate_thumbnail(video_path, moments[0])
```

Ainsi mon programme génère automatiquement une miniature du moment le plus intense.
J'ai mis une nouvelle vidéo pour le test.mp4, ma vidéo dure 1min29.
J'avais une erreur lorsque je lançais mon programme.
L'erreur venait du fait que le programme essayait de récupérer trois valeurs(start, end, score) dans chaque moment, alors que la liste utilisée n'en contenait que deux (start, end). La boucle a donc été corrigée pour ne lire que ces deux valeurs, ce qui correspond aux données réellement stockées dans la liste.

De plus j'ajoute une petite modification dans ma fonction, actuellement j'ai la première frame du meilleur moment mais ce n'est pas forcément la meilleure image de la période sur laquelle est définie le meilleur moment.

Par exemple: moment fort : 30.96s → 32.53s
La meilleur frame peut être : 31.84s

Je change ma fonction en y ajoutant ces éléments :

```
def generate_thumbnail(video_path, moment, motion_array):

    start, end, score = moment

    # frame avec le score maximum dans le segment
    best_frame = start + np.argmax(motion_array[start:end])
```

```
    cap = cv2.VideoCapture(video_path)
    cap.set(cv2.CAP_PROP_POS_FRAMES, int(best_frame))

    ret, frame = cap.read()

    if ret:
        output_path = Path("outputs/thumbnail.jpg")
        cv2.imwrite(str(output_path), frame)
        print("🖼️ Miniature générée :", output_path)

    cap.release()
```

J’ajoute motion_array car il contient le score de chaque frame et on l’utilise pour trouver `np.argmax(motion_array[start:end])`, c’est à dire la frame avec le mouvement maximum.

Semaine 6

Ré-adaptation de la génération du best-of

Un première sélection est réalisée en ajoutant des segments dans le best-of jusqu'à atteindre deux conditions :

- Un nombre minimal de segments (au moins 5 moments forts)
- Une durée minimale du best-of correspondant à 20 pourcent de la durée totale de la vidéo

Cette approche permet de garantir que le best-of contienne suffisamment de contenu intéressant.

```
video_duration = frame_count / fps



    min_bestof_duration = video_duration * 0.2
    min_moments = 5
```

```
    selected_moments = []
    current_duration = 0

    # 1. On ajoute d'abord les moments forts
    for start, end, score in moments:
        segment_duration = (end - start) / fps

        selected_moments.append((start, end))
        current_duration += segment_duration

        if len(selected_moments) >= min_moments and
current_duration >= min_bestof_duration:
            break
```

Complément avec les segments de scènes

Dans certains cas, les moments forts détectés ne suffisent pas à atteindre la durée minimale fixée. Pour résoudre ce problème, l'algorithme utilise également les coupures de scènes détectées précédemment dans la vidéo. Ces coupures permettent de créer des segments supplémentaires qui complètent le best-of jusqu'à atteindre la durée minimale requise.

```
 if current_duration < min_bestof_duration:
        for start, end in scene_segments:

            # éviter d'ajouter un segment qui chevauche déjà un moment
sélectionné
            overlap = False
            for sel_start, sel_end in selected_moments:
                if not (end < sel_start or start > sel_end):
                    overlap = True
                    break

            if not overlap:
                selected_moments.append((start, end))
                current_duration += (end - start) / fps

            if current_duration >= min_bestof_duration:
             break
```

Réorganisation chronologique des segments

Les segments sélectionnées sont initialement triés selon leur score d'intensité. Cependant, pour assurer une cohérence temporelle lors du visionnage, les segments retenus sont ensuite ordonnés selon leur position dans la vidéo. Cette étape garantit que le best-of suit une progression chronologique naturelle.

Tri chronologique pour garder un best-of cohérent

```
selected_moments = sorted(selected_moments, key=lambda x: x[0])
```

Fusion des segments proches

Une étape supplémentaire de post-traitement a été ajoutée afin d'améliorer la fluidité du montage. Lorsque deux segments sélectionnés sont séparés par un intervalle très court, ils sont fusionnés en un seul segment plus long. Cette méthode permet d'éviter les micro-coupures trop rapprochées qui pourrait rendre le best of difficile a suivre visuellement.

On crée une fonction

```
def merge_close_segments(segments, max_gap_frames):
    if not segments:
        return []

    # segments triés par début
    segments = sorted(segments, key=lambda x: x[0])

    merged = [segments[0]]

    for current_start, current_end in segments[1:]:
        last_start, last_end = merged[-1]

        # si l'écart entre les deux segments est petit, on fusionne
        if current_start - last_end <= max_gap_frames:
            merged[-1] = (last_start, max(last_end, current_end))
        else:
            merged.append((current_start, current_end))
```

```
    return merged

      #  Fusion des segments trop proches

    merge_gap_seconds = 1.0
    merge_gap_frames = int(merge_gap_seconds * fps)
    selected_moments = merge_close_segments(selected_moments,
merge_gap_frames)

    #  Recalcul de la durée après fusion

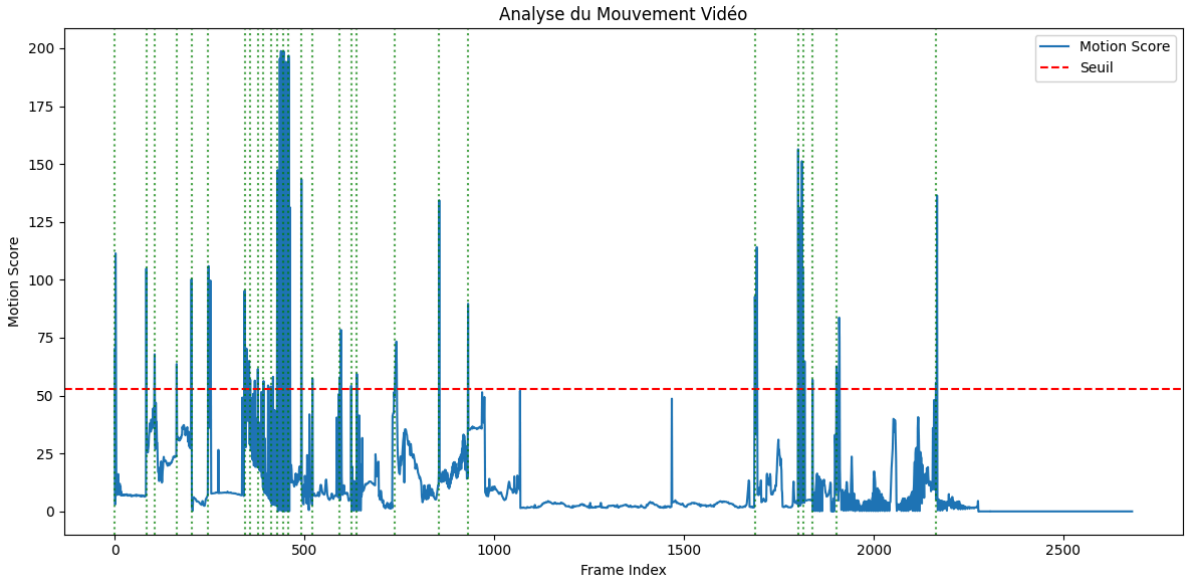
    current_duration = 0
    for start, end in selected_moments:
        current_duration += (end - start) / fps
```

Pour la semaine 7

Mise en place de la détection audio et fusion de l’analyse audio et vidéo.

Annexe

Cyriac Dupouey




```
frame_index,time_seconds,motion_score
0,0.0,70.07414582240506
83,2.769433333333333,104.86926943179328
105,3.5035000000000003,67.86132093521529
163,5.438766666666667,63.65187572838946
202,6.740066666666666,100.25744664102547
246,8.2082,67.06260987775377
342,11.4114,95.23926679432222
357,11.911900000000001,56.943973877403295
377,12.579233333333333,61.5116222216844
392,13.079733333333333,56.28290073824151
411,13.713700000000001,53.95905079203564
428,14.280933333333333,98.37685101624032
443,14.781433333333334,186.93746731428337
457,15.248566666666667,98.27229554645439
492,16.4164,143.35005997694614
521,17.384033333333335,57.35935619847965
593,19.786433333333335,58.00558427299443
623,20.787433333333333,54.251013288887954
638,21.287933333333335,59.30677758598489
738,24.6246,53.717034155753154
855,28.5285,134.3170844034808
931,31.06436666666667,89.67736890589438
1687,56.289566666666666,92.73768660919953
1801,60.09336666666667,156.3963498085893
1816,60.59386666666667,61.01877987628943
1839,61.3613,56.84148663807952
1902,63.4634,62.6898975457934
2164,72.20546666666667,55.577812877760024
```

Miniature

